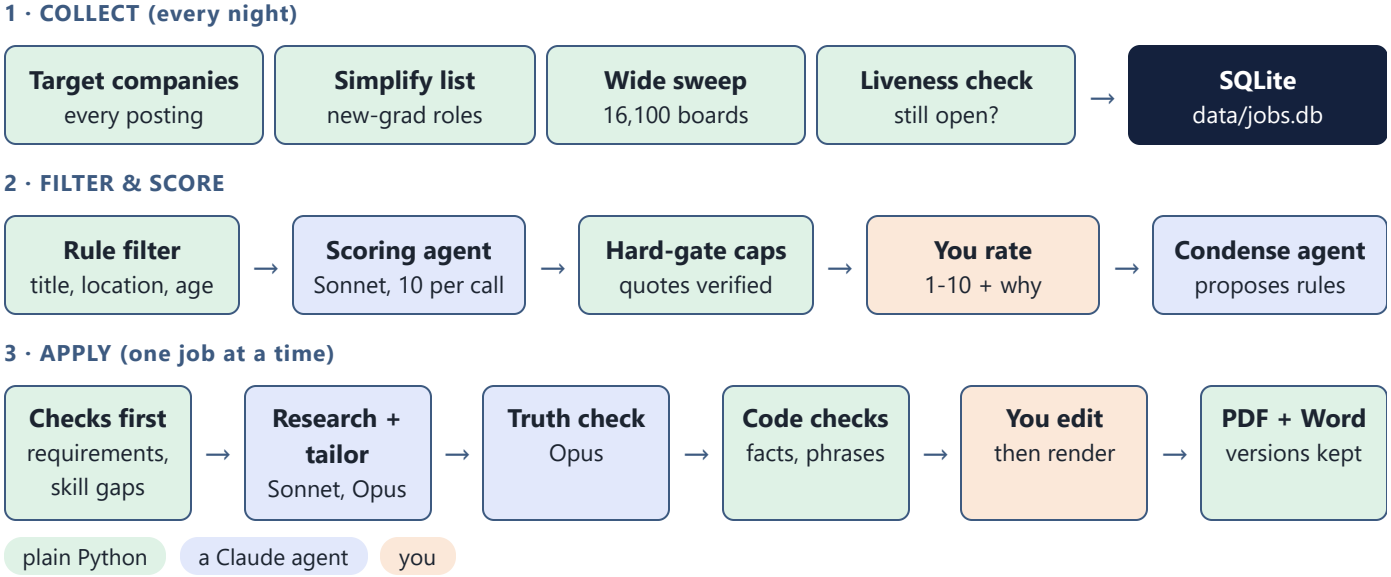
Job Sourcer

How the system works as of October 26, 2026: where jobs come from, how they're filtered and scored, how applications are drafted, and the rules that keep it honest.

Job Sourcer is a personal job-search tool that runs on one PC. Every night it collects new-grad postings from public job boards, drops obvious mismatches with cheap rules, and has Claude score the rest against the candidate's background. From any job it drafts a tailored one-page resume (and, when wanted, a cover letter) as PDF and Word files, written only from facts in the candidate's own profile. A local web app ties it together: browsing and rating jobs, drafting, tracking applications, and keeping track of contacts.

6,659	1,458	2,822	16,100	30
jobs collected	scored by Claude	filtered out by rules	job boards in the sweep	application drafts

The big picture



Three ideas that hold the design together

- **Cheap work before expensive work.** Anything a keyword or a date can decide is decided in Python before Claude sees the job, so the Claude plan is spent only on real candidates.
- **Never trust the model blindly.** Every agent returns JSON that Python validates, and whatever code can check is checked in code: a hard requirement only counts if its quote is found word for word in the posting, and a resume skill only stays if it's in the profile.
- **The candidate stays in charge.** Agents can propose scoring rules but can't write them, nothing is sent anywhere automatically, and only letters the candidate wrote entirely themselves count as their writing voice.

1 · Collecting jobs

Every source turns its postings into one common `Job` shape (source, company, job id, title, location, link, description, posting date). Saving is an upsert keyed on source + company + job id, so a job seen twice is updated, never duplicated. A job that disappears from its board is marked closed but keeps its score and history.

Source	What it does	Politeness
Target companies `targets.py`	Pulls every open posting at a short list of companies in `companies.yaml`, whatever the title or date: through a board API, the company's own page, Oracle's public JSON API, or headless Edge for a site that blocks scripts. Target jobs get a ★ and are scored first.	Detail pages fetched only for new titles that pass the filter. An empty result never closes jobs.
Simplify list `import_feed.py`	Imports the community SimplifyJobs new-grad list. Postings on Greenhouse, Lever or Ashby get their full description; the rest are stored as listing-only.	Small delay between requests; later runs fetch only new listings.
Wide sweep `sweep.py`	Checks about 16,100 public boards for postings from the last 7 days whose title has a sweep keyword (new grad, junior, data analyst, intern...).	8 boards at a time. Boards that 404 are skipped for 30 days.
Moved boards `find_moved_boards.py`	A dead board often means the company switched job systems. This tries each dead board's name on the other two systems; 238 boards were recovered this way.	About 2 requests per dead board, run occasionally.
Liveness `liveness.py`	Asks the job system whether the best and tracked jobs are still open, tracked ones first.	One request per job; network errors never close anything.

2 · Filtering and cleanup (no AI)

Rule	How it works
Title keywords	Whole-word, case-insensitive matches from `filters.yaml`: a target word must appear ("engineer", "data", "analyst"...), and seniority or wrong-field words ("senior", "staff", "sales"...) drop the job.
Location	A job is dropped only if *every* location it lists is excluded. A place that's clearly in the US (a state code or "United States") always passes, so "Vienna, VA" isn't mistaken for Austria.
Age	Postings more than 14 days old move to their own tab and aren't scored, so no Claude usage goes to jobs that are probably filled. Target-company jobs are exempt.
Duplicates	Copies of one posting (same company, title and description, often one per city) are grouped; only one is scored and its score is copied to the rest.

Priority	A points guess (entry-level words up, "3+ years" down, target companies first) sets the scoring order, so the nightly cap goes to the best bets. Priority never removes a job.

3 · The agents

An "agent" is one Claude call with one job. All of them go through `scorer.call_claude`, which runs Claude Code in print mode (`claude -p`) on the user's Claude plan, so there is no API key anywhere. Each call is locked down: no tools (`--tools ""`), no project settings (`--safe-mode`), JSON output, no saved session. The only exception is company research, which may use web search and fetch and nothing else. Every call is logged with its tokens and time.

Agent	Model	Job	What checks it
Scorer	Sonnet	Scores 10 jobs per call 1-10 for fit, with a reason, matched skills, gaps, and any hard requirements quoted from the posting.	Python validates every field, verifies each quote, applies the caps.
Condense	default	Turns ratings that disagree with the scorer into proposed scoring rules.	Writes a proposal only; the user copies what they agree with.
Skill check	Sonnet	Before drafting, lists skills the posting wants that the profile doesn't show, so the user can say whether they have them.	Python drops any whose quote isn't in the posting or that were answered before.
Research	Sonnet	Finds the company's mission, values and products, each with a source link and the company's exact words.	Python downloads each page and drops facts whose quote isn't on it.
Tailor	Opus	Writes the resume and (optionally) the letter from the profile and the posting.	Shape validated; then the truth check and code checks.
Truth check	Opus	A second, separate call that compares every claim with the profile and fixes anything invented, overstated or unconfirmed.	Its fixes are listed for the user to review.
Reviewer / Reviser	Opus	Critiques a letter on request, then rewrites only the letter with the feedback the user keeps.	The revision is truth-checked again.

4 · How a job is scored

The scorer reads the candidate's short profile, their approved feedback rules and a rubric. Some requirements rule a candidate out however good the fit is; the scorer reports these as **hard gates**, each with an exact quote. Python searches the posting for the quote, and **only a verified quote caps the score**. Claude scores the fit as if the gates weren't there (the caps live only in code), so `fit_score` is the honest fit and `score` is after caps.

Gate	Max score
Required 5+ years of experience	3

Required 3-4 years	5
Clearance, required credential, people management, non-English language, non-US location	4
Current students or later graduates only	2

"Preferred" or "nice to have" never counts, and neither do ranges that start at 2 years or fewer, years with an alternative ("or a Master's"), or years that count internships.

Re-checks. Every score is stamped with the prompt version (now v6) plus a fingerprint of the rubric, instructions, profile and feedback rules. When any of them changes, good jobs scored under the old stamp are re-scored, 20 a night.

The feedback loop



The rating queue mixes in low scores on purpose, so jobs the scorer under-rated get caught too.

5 · Drafting an application

1. **Requirements check.** If scoring found a verified requirement the candidate can't meet, the posting's own words are shown first, and drafting needs a deliberate "draft anyway".
2. **Skill check.** The candidate answers yes or no to skills the posting wants that the profile doesn't show. A yes, in their own words, is added to the profile; a no is remembered and never asked again.
3. **Research** (letters only), **tailor**, **truth check**, then **code checks**.
4. **Edit** any line in the app, then **Save & generate**: HTML templates printed to PDF by headless Edge (stepping the font down to fit one page), plus a Word file built with python-docx in a single column for ATS parsers.
5. **Versions.** Each draft, revision and render is saved. Marking a job applied copies the exact files sent into a dated folder, and the app shows what was changed by hand from the generated draft.

The resume

Header, a one-sentence summary, Education, Projects (the main section: the two most relevant get 2-3 bullets, a tools line and a repository link), Experience, and Skills as labeled lines. Job titles, dates, the education line and project names come from a "canonical headings" block in the profile and print identically on every resume; drift is corrected automatically.

The cover letter (optional; resume-only is the default)

¶	What it does
1	The role, and the single strongest piece of evidence for its main requirement.
2	One story told as what was checked, what was found, and what changed.
3	One specific tie to the company, then logistics (relocation for on-site roles), and stop.

250-350 words. Requirements are shown through evidence, never echoed back from the posting. The drafter sees the openings and closings of recent letters and may not reuse them; a code check flags any 8-word phrase repeated from another letter.

6 · Rules, and how each is enforced

Code rules hold every time. Prompt rules are instructions Claude follows most of the time, so the important ones are also checked by a second agent or by the user.

Rule	Enforced by	How
Truthfulness		
Only facts from the profile; nothing invented or overstated	Prompt ×2	Tailor rule, then the truth check audits every claim.
Unconfirmed facts stay out	Prompt ×2	Anything marked `[confirm]` in the profile is skipped.
Every resume skill exists in the profile	Code	Whole-term match; missing skills are removed and reported.
Fixed headings never drift	Code	Canonical headings restored on every draft and save.
Writing		
No em dashes, AI-isms or stock closers	Prompt + Code	Banned-phrase regexes and an em dash count.
Letter: 3 paragraphs, 250-350 words, no repeats across letters	Prompt ×3 + Code	Tailor, truth check and reviewer; length and 8-word repeat checks in code.
Voice only from the candidate's own letters	Prompt + You	AI-written text is never saved as a voice sample.
Resume fits one page	Prompt + Code	Length limits in the prompt; the page count is read from the PDF.
Scoring and collection		
Hard requirements cap the score	Code	Quote verified in the posting, then capped.
Agents can't change the scoring rules	Code + You	Condense writes a proposal file only.
The nightly run can't drain the Claude plan	Code	60 jobs a night; old postings skipped; usage logged per call.
Be polite to job boards	Code	Delays, 8 boards at a time, dead boards skipped.
No API keys or personal data in the repository	Code	Claude runs through the logged-in CLI; profiles, applications and the database are gitignored.

7 · Tracking applications and contacts

- **Status:** Reached out → Applied → Interviewing → Offer or Rejected, each with a note and a full history. Applied jobs leave the working lists.
- **Referral reminders:** "Reached out for a referral" starts a 2-day clock. If the job still isn't marked applied, a Windows notification (checked hourly), a banner in the app, and a line in the nightly notification say it's time to apply directly.
- **People:** contacts and where they work. Jobs at a contact's company show their name in the job list and on the job's page.

8 · The nightly run

1 Targets a minute or two → **2 Simplify** seconds → **3 Sweep** 15-20 minutes → **4 Liveness** under a minute → **5 Score** up to 60 jobs

Windows Task Scheduler starts `nightly.py` at 2 AM. It wakes the PC and keeps it awake until the run ends, a lock file stops runs overlapping (and stops the app scoring at the same time), and one failing step doesn't stop the others. The closing notification reports new jobs, how many scored 7+, failures and the backlog, and says plainly when something broke quietly: an expired Claude login, or far fewer boards or jobs than the last 7 nights.

9 · The web app and where things live

`start.bat` starts a Flask server that only listens on the local machine. Long tasks run in background threads while the page polls for progress. **After any change to a `.py` file, restart it:** the running server keeps the old code (templates are re-read from disk, so a mismatch shows up as a garbled document).

Page	What it's for
Jobs	Tabs: Scored, To score, ★ Target companies, Filtered out, Listing only, Older than 14 days, Closed, Applied & reached out.
Rate · Score · Feedback	Rate jobs; run and watch scoring; propose and approve scoring rules.
Fetch	Imports, sweep, target and liveness checks, the last nightly run, Claude usage.
Applications · People	Drafts, the editor, versions and files; contacts.

File	Role
`profile.md` / `full-profile.md`	Short profile the scorer reads / master profile the drafter reads (both gitignored).
`filters.yaml`, `feeds.yaml`, `companies.yaml`, `nightly.yaml`	Filter rules, sweep keywords, target companies, nightly steps and caps.
`data/jobs.db`	SQLite: jobs, ratings, applications and their versions, status history, contacts, skill answers, Claude usage.

`applications/`	Generated PDF and Word files, with copies of what was sent (gitignored).